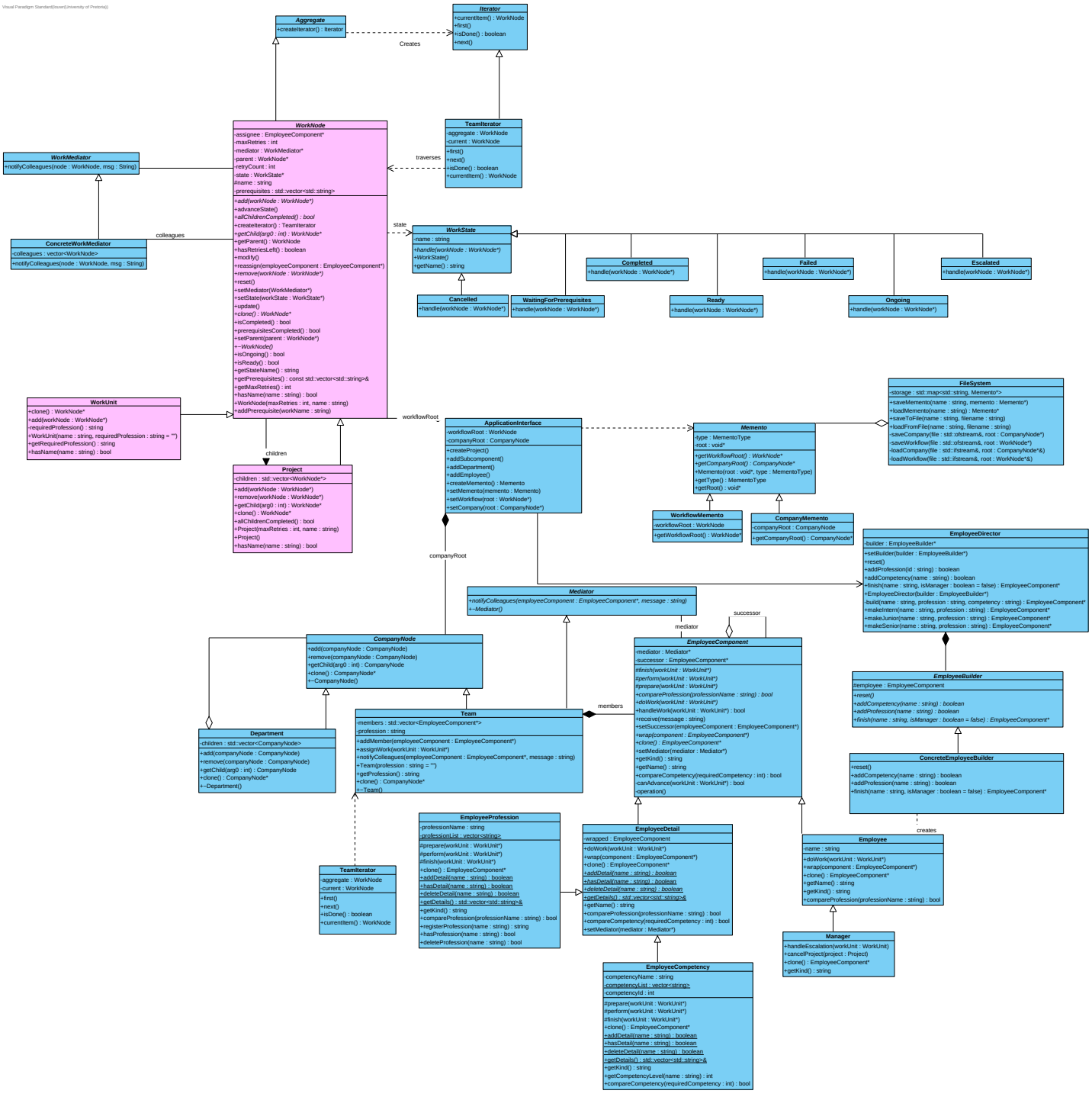
Visual Paradigm Standard(louw(University of Pretoria))
Aggregate
+createIterator() : Iterator
Creates
Iterator
+currentItem() : WorkNode
+first()
+isDone() : boolean
+next()
WorkNode
-assignee : EmployeeComponent*
-maxRetries : int
-mediator : WorkMediator*
-parent : WorkNode*
-retryCount : int
-state : WorkState*
#name : string
-prerequisites : std::vector<std::string>
+add(workNode : WorkNode*)
+advanceState()
+allChildrenCompleted() : bool
+createIterator() : TeamIterator
+getChild(arg0 : int) : WorkNode*
+getParent() : WorkNode
+hasRetriesLeft() : boolean
+modify()
+reassign(employeeComponent : EmployeeComponent*)
+remove(workNode : WorkNode*)
+reset()
+setMediator(WorkMediator*)
+setState(workState : WorkState*)
+update()
+clone() : WorkNode*
+isCompleted() : bool
+prerequisitesCompleted() : bool
+setParent(parent : WorkNode*)
+~WorkNode()
+isOngoing() : bool
+isReady() : bool
+getStateName() : string
+getPrerequisites() : const std::vector<std::string>&
+getMaxRetries() : int
+hasName(name : string) : bool
+WorkNode(maxRetries : int, name : string)
+addPrerequisite(workName : string)
TeamIterator
-aggregate : WorkNode
-current : WorkNode
+first()
+next()
+isDone() : boolean
+currentItem() : WorkNode
traverses
WorkMediator
+notifyColleagues(node : WorkNode, msg : String)
ConcreteWorkMediator
-colleagues : vector<WorkNode>
+notifyColleagues(node : WorkNode, msg : String)
colleagues
state
WorkState
-name : string
+handle(workNode : WorkNode*)
+WorkState()
+getName() : string
Cancelled
+handle(workNode : WorkNode*)
WaitingForPrerequisites
+handle(workNode : WorkNode*)
Completed
+handle(workNode : WorkNode*)
Ready
+handle(workNode : WorkNode*)
Failed
+handle(workNode : WorkNode*)
Ongoing
+handle(workNode : WorkNode*)
Escalated
+handle(workNode : WorkNode*)
WorkUnit
+clone() : WorkNode*
+add(workNode : WorkNode*)
-requiredProfession() : string
+WorkUnit(name : string, requiredProfession : string = "")
+getRequiredProfession() : string
+hasName(name : string) : bool
children
Project
-children : std::vector<WorkNode*>
+add(workNode : WorkNode*)
+remove(workNode : WorkNode*)
+getChild(arg0 : int) : WorkNode*
+clone() : WorkNode*
+allChildrenCompleted() : bool
+Project(maxRetries : int, name : string)
+Project()
+hasName(name : string) : bool
workflowRoot
ApplicationInterface
-workflowRoot : WorkNode
-companyRoot : CompanyNode
+createProject()
+addSubcomponent()
+addDepartment()
+addEmployee()
+createMemento() : Memento
+setMemento(memento : Memento)
+setWorkflow(root : WorkNode*)
+setCompany(root : CompanyNode*)
companyRoot
Memento
-type : MementoType
-root : void*
+getWorkflowRoot() : WorkNode*
+getCompanyRoot() : CompanyNode*
+Memento(root : void*, type : MementoType)
+getType() : MementoType
+getRoot() : void*
WorkflowMemento
-workflowRoot : WorkNode
+getWorkflowRoot() : WorkNode*
CompanyMemento
-companyRoot : CompanyNode
+getCompanyRoot() : CompanyNode*
FileSystem
-storage : std::map<std::string, Memento*>
+saveMemento(name : string, memento : Memento*)
+loadMemento(name : string) : Memento*
+saveToFile(name : string, filename : string)
+loadFromFile(name : string, filename : string)
-saveCompany(file : std::ofstream&, root : CompanyNode*)
-saveWorkflow(file : std::ofstream&, root : WorkNode*)
-loadCompany(file : std::ifstream&, root : CompanyNode*&)
-loadWorkflow(file : std::ifstream&, root : WorkNode*&)
EmployeeDirector
-builder : EmployeeBuilder*
+setBuilder(builder : EmployeeBuilder*)
+reset()
+addProfession(id : string) : boolean
+addCompetency(name : string) : boolean
+finish(name : string, isManager : boolean = false) : EmployeeComponent*
+EmployeeDirector(builder : EmployeeBuilder*)
-build(name : string, profession : string, competency : string) : EmployeeComponent*
+makeIntern(name : string, profession : string) : EmployeeComponent*
+makeJunior(name : string, profession : string) : EmployeeComponent*
+makeSenior(name : string, profession : string) : EmployeeComponent*
Mediator
+notifyColleagues(employeeComponent : EmployeeComponent*, message : string)
+~Mediator()
mediator
successor
CompanyNode
+add(companyNode : CompanyNode)
+remove(companyNode : CompanyNode)
+getChild(arg0 : int) : CompanyNode
+clone() : CompanyNode*
+~CompanyNode()
EmployeeComponent
-mediator : Mediator*
-successor : EmployeeComponent*
#finish(workUnit : WorkUnit*)
#perform(workUnit : WorkUnit*)
#prepare(workUnit : WorkUnit*)
+compareProfession(professionName : string) : bool
+doWork(workUnit : WorkUnit*)
+handleWork(workUnit : WorkUnit*) : bool
+receive(message : string)
+setSuccessor(employeeComponent : EmployeeComponent*)
+wrap(component : EmployeeComponent*)
+clone() : EmployeeComponent*
+setMediator(mediator : Mediator*)
+getKind() : string
+getName() : string
+compareCompetency(requiredCompetency : int) : bool
-canAdvance(workUnit : WorkUnit*) : bool
-operation()
EmployeeBuilder
#employee : EmployeeComponent
+reset()
+addCompetency(name : string) : boolean
+addProfession(name : string) : boolean
+finish(name : string, isManager : boolean = false) : EmployeeComponent*
ConcreteEmployeeBuilder
+reset()
+addCompetency(name : string) : boolean
+addProfession(name : string) : boolean
+finish(name : string, isManager : boolean = false) : EmployeeComponent*
creates
Department
-children : std::vector<CompanyNode>
+add(companyNode : CompanyNode)
+remove(companyNode : CompanyNode)
+getChild(arg0 : int) : CompanyNode
+clone() : CompanyNode*
+~Department()
Team
-members : std::vector<EmployeeComponent*>
-profession : string
+addMember(employeeComponent : EmployeeComponent*)
+assignWork(workUnit : WorkUnit*)
+notifyColleagues(employeeComponent : EmployeeComponent*, message : string)
+Team(profession : string = "")
+getProfession() : string
+clone() : CompanyNode*
+~Team()
members
TeamIterator
-aggregate : WorkNode
-current : WorkNode
+first()
+next()
+isDone() : boolean
+currentItem() : WorkNode
EmployeeProfession
-professionName : string
-professionList : vector<string>
#prepare(workUnit : WorkUnit*)
#perform(workUnit : WorkUnit*)
#finish(workUnit : WorkUnit*)
+clone() : EmployeeComponent*
+addDetail(name : string) : boolean
+hasDetail(name : string) : boolean
+deleteDetail(name : string) : boolean
+getDetails() : std::vector<std::string>&
+getKind() : string
+compareProfession(professionName : string) : bool
+registerProfession(name : string) : string
+hasProfession(name : string) : bool
+deleteProfession(name : string) : bool
EmployeeDetail
-wrapped : EmployeeComponent
+doWork(workUnit : WorkUnit*)
+wrap(component : EmployeeComponent*)
+clone() : EmployeeComponent*
+addDetail(name : string) : boolean
+hasDetail(name : string) : boolean
+deleteDetail(name : string) : boolean
+getDetails() : std::vector<std::string>&
+getName() : string
+compareProfession(professionName : string) : bool
+compareCompetency(requiredCompetency : int) : bool
+setMediator(mediator : Mediator*)
EmployeeCompetency
-competencyName : string
-competencyList : vector<string>
-competencyId : int
#prepare(workUnit : WorkUnit*)
#perform(workUnit : WorkUnit*)
#finish(workUnit : WorkUnit*)
+clone() : EmployeeComponent*
+addDetail(name : string) : boolean
+hasDetail(name : string) : boolean
+deleteDetail(name : string) : boolean
+getDetails() : std::vector<std::string>&
+getKind() : string
+getCompetencyLevel(name : string) : int
+compareCompetency(requiredCompetency : int) : bool
Employee
-name : string
+doWork(workUnit : WorkUnit*)
+wrap(component : EmployeeComponent*)
+clone() : EmployeeComponent*
+getName() : string
+getKind() : string
+compareProfession(professionName : string) : bool
Manager
+handleEscalation(workUnit : WorkUnit)
+cancelProject(project : Project)
+clone() : EmployeeComponent*
+getKind() : string